EXERCISE 1 — ELECTRONICS STORE INVENTORY SYSTEM

Question 1: Create an Interface [2 Points]

Requirements:

- Create an interface `IElectronicDevice`. **(1 Point)**
- Define the following methods: **(1 Point)**

Return Type	Method Name	Description
`void`	`inputDevice()`	Input details of a device
`void`	`displayDevice()`	Display all details of the device

Question 2: Create the `ElectronicDevice` Class [6 Points]

Requirements:

Create a class `ElectronicDevice` implementing `IElectronicDevice`.

Fields:

- `String name`
- `String brand`
- `int releaseYear`
- `double price`

Constructors:

- No-argument constructor **(0.5 Point)**
- Constructor with all fields **(0.5 Point)**

Getters and Setters:

- Standard getters and setters for all fields **(1 Point)**

Implement Methods:

- `inputDevice()` — use `BufferedReader` to input and validate: **(1 Point)**
  - `name`, `brand` ≥ 2 characters
  - `releaseYear` ≥ 2000 and ≤ current year
  - `price` > 0
- `displayDevice()` — nicely formatted output **(1 Point)**
- `isPremium()` — returns `true` if `price` > 1000.0 **(1 Point)**
- Inside `displayDevice()`, show: **(0.5 Point)**
  - `"Device '[Name]' is a premium product."`
  - OR `"Device '[Name]' is standard."`

- Call `isPremium()` and show the result in `displayDevice()` **(0.5 Point)**

Question 3: Create the `InventoryManager` Class [5 Points]

Requirements:

Create class `InventoryManager`.

Manage:

- A list of `ElectronicDevice` objects via `ArrayList`

Methods:

Method Name	Description	Points
`addDevice()`	Add a new device (calls `inputDevice()`)	1
`showAllDevices()`	Display all devices	1
`findMostExpensive()`	Find and show the most expensive device	3

Question 4: Create the `Main` Class [3 Points]

Requirements:

Create `Main` class.

In `main()`:

- Show menu: **(1.5 Points)**

```
=== ELECTRONICS STORE MENU ===
1. Add New Device
2. Show All Devices
3. Find Most Expensive Device
4. Exit
Your choice:
```

- Call appropriate methods from `InventoryManager` **(1.5 Points)**
- End the program on option 4

Question 5: Exception Handling in `inputDevice()` [4 Points]

Requirements:

Improve input validation in `inputDevice()` using a custom exception.

Tasks:

1. Create a new exception class `InvalidDeviceDataException`, extending `Exception`. **(1 Point)**
2. In `inputDevice()`, throw the custom exception when any field is invalid. **(1 Point)**

3. Catch exceptions in a loop to allow retry until input is valid. **(1 Point)**
4. Show meaningful error messages for each validation failure. **(1 Point)**

EXERCISE 2 — COURSE MANAGEMENT SYSTEM

Question 1: Abstract Class for Course

a) Create the abstract class `Course`

Tasks

Create an abstract class named `Course`.

The class should contain the following attributes:

- `courseId` (`String`)
- `courseName` (`String`)
- `instructor` (`String`)
- `startDate` (`String`)
- `maxStudents` (`int`)

Requirements

- Create a no-argument constructor.
- Create a parameterized constructor.
- Provide getter and setter methods for all attributes.
- Define the following abstract methods:

```
public abstract void input(BufferedReader reader) throws IOException;
public abstract void display();
public abstract String getCourseType();
```

Question 2: `CostCalculable` Interface

a) Create the `CostCalculable` interface

Tasks

Create an interface named `CostCalculable`.

Requirements

Declare the following method:

```
double getCostMetric();
```

Question 3: `OnlineCourse` Class

a) Create the `OnlineCourse` class

Tasks

Create a class named `OnlineCourse`, inheriting from the `Course` class and implementing the `CostCalculable` interface.

Add the following attributes:

- `platform` (`String`)
- `courseFee` (`double`)

Requirements

- Create a no-argument constructor.
- Create a full constructor (including inherited attributes and new attributes), using the `super` keyword.
- Provide getter and setter methods for the new attributes.
- Override the abstract methods inherited from `Course`:
  - `input()` must prompt the user to enter all `OnlineCourse` information using `BufferedReader`.
  - `display()` must display all information of an `OnlineCourse` object.
  - `getCourseType()` must return the string `"ONLINE"`.
- Implement the method inherited from `CostCalculable`:
  - `getCostMetric()` must return the course fee.

Question 4: `OfflineCourse` Class

a) Create the `OfflineCourse` class

Tasks

Create a class named `OfflineCourse`, inheriting from the `Course` class and implementing the `CostCalculable` interface.

Add the following attributes:

- `roomName` (`String`)
- `hasLab` (`boolean`)

Requirements

- Create a no-argument constructor.
- Create a full constructor using the `super` keyword.
- Provide getter and setter methods for the new attributes.
- Override the abstract methods inherited from `Course`:

- `input()` must prompt the user to enter all `OfflineCourse` information using `BufferedReader`.
- `display()` must display all information of an `OfflineCourse` object.

b) Estimated Cost Calculation for Offline Courses

Tasks

In the `OfflineCourse` class, create the following method:

```
public double calculateEstimatedCost();
```

Cost calculation rules

- If `hasLab == true`: 200,000 VND per student
- If `hasLab == false`: 120,000 VND per student

Estimated cost = `maxStudents` × cost per student

- After calling the `display()` method, also display the estimated course cost.
- Implement the method inherited from `CostCalculable`:
  - `getCostMetric()` must return the estimated course cost calculated by `calculateEstimatedCost()`.
- Implement the remaining abstract method inherited from `Course`:
  - `getCourseType()` must return the string `"OFFLINE"`.

Question 5: Menu System and Program Testing

a) Data storage structure

Tasks

Use a single `ArrayList<Course>` to store all types of courses:

```
ArrayList<Course> courseList = new ArrayList<>();
```

Do not create separate lists for `OnlineCourse` or `OfflineCourse`.

b) Main menu

Tasks

Create a class named `CourseManagementTest` containing the `main` method to implement a menu-driven program.

The program must use `BufferedReader` for user input and provide the following options:

```
===== COURSE MANAGEMENT SYSTEM =====
1. Add an Online Course
2. Add an Offline Course
3. Display all Courses
4. Display Online Courses (sorted by cost in ascending order)
```

```
5. Display Offline Courses (sorted by cost in ascending order)
6. Exit
Your choice:
```

c) Menu functionality

Tasks

- When the user selects option (1), create an `OnlineCourse` object, call `input()`, and add it to `courseList`.
- When the user selects option (2), create an `OfflineCourse` object, call `input()`, and add it to `courseList`.
- When the user selects option (3), display all courses in `courseList` by calling `display()`.
- When the user selects option (4), filter `OnlineCourse` objects from `courseList`, sort them by `getCostMetric()` in ascending order, and display them.
- When the user selects option (5), filter `OfflineCourse` objects from `courseList`, sort them by `getCostMetric()` in ascending order, and display them.
- When the user selects option (6), exit the program.